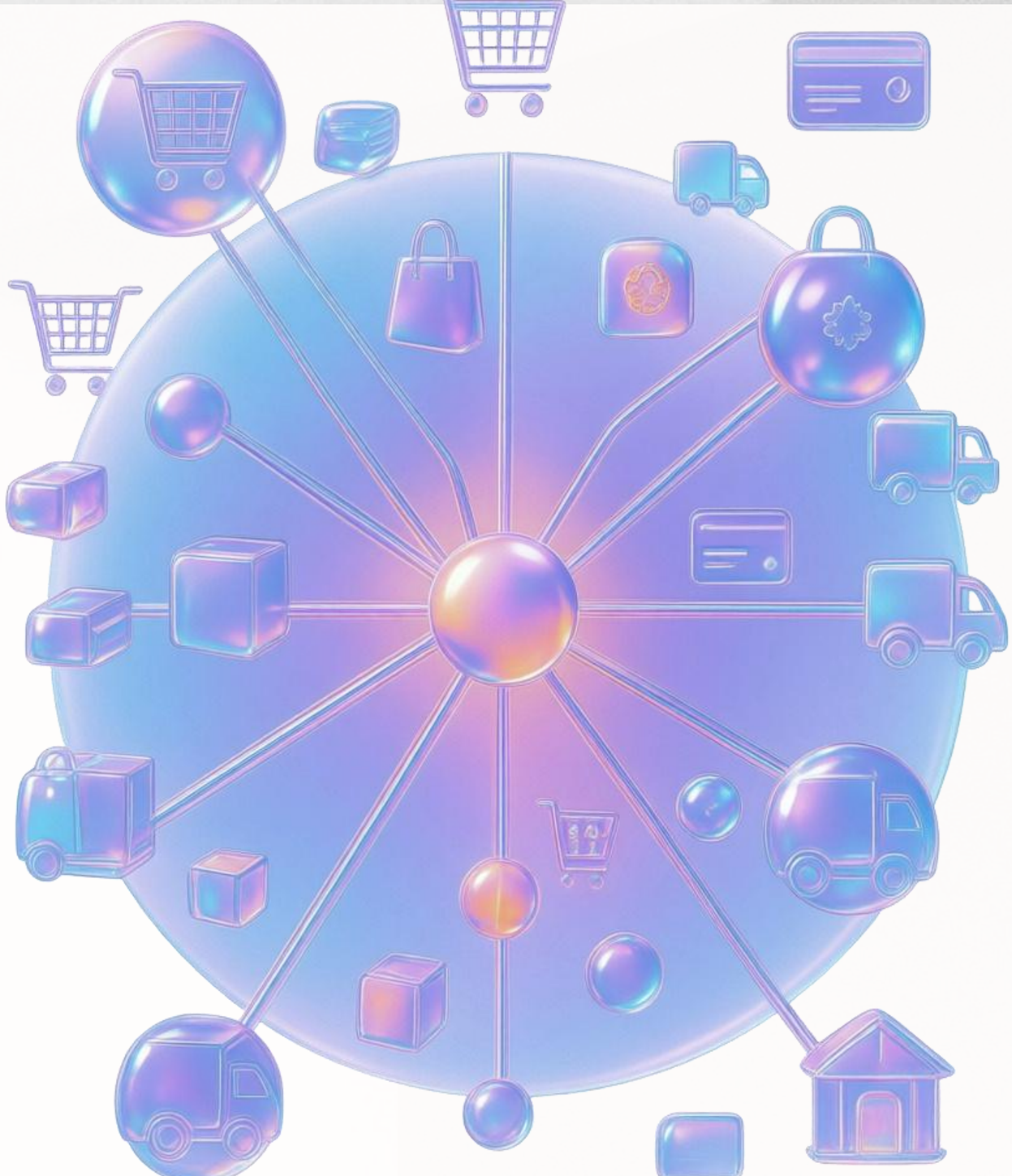
ShopTrace

Architectural Deep Dive into Modern Software Development

Ayşe Sümeyra MERT
212010020004
Büşra TOPDEMİR 212010020051

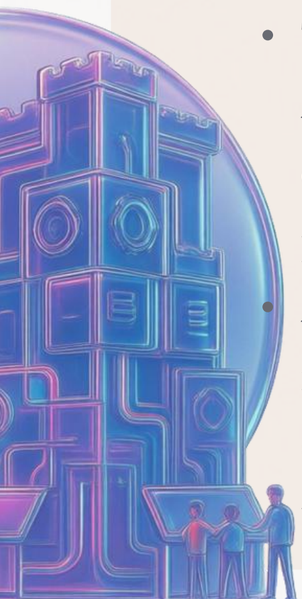


Monolithic versus Microservice Architecture

Monolithic Architecture

The entire application is encapsulated within a single JAR: User, Order, and Payment functionalities operate within a unified JAR/WAR file. This approach facilitates a rapid initiation; however, it becomes increasingly unwieldy as it expands.

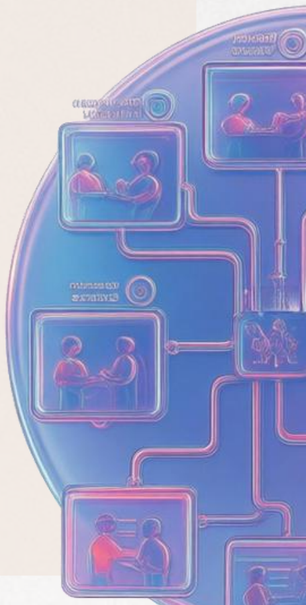
- The components are closely interconnected. A minor alteration in one module can impact other modules, extending the testing processes.
- A singular point of failure has the potential to incapacitate the entire system. Independent deployment or selection of technology is unfeasible.



Microservices Architecture

Each service (User, Order, Payment) is developed, packaged, and deployed autonomously. Each possesses its own codebase and configuration.

- Services interact exclusively through clearly defined APIs. Components are loosely coupled, ensuring that changes remain isolated.
- Each service is independently scalable. If the Order Service experiences high traffic, only that service would be scaled, while the other services would remain unaffected.



ShopTrace Project Synopsis

The e-commerce platform is constructed upon three fundamental services and a completely traceable infrastructure.

User Support

PORT: 8081

Authentication and user profile management verify the user's presence and authorization in incoming requests. This process is invoked in succession by the Order Service and functions without direct user interaction. The /user endpoint returns a User OK response.

Ordering Service

PORT: 8082

It serves as the primary orchestrator of the system. Upon the user's click of the 'Complete Order' button, this service is initiated; it sequentially invokes the User and Payment services from within the code, forming a cohesive chain of operations. It consolidates and returns all results.

Payment Solution

PORT: 8083

Payment processing and verification service. It processes card information and makes approval or rejection decisions. It responds to requests from the Order Service and returns the outcome. The /payment endpoint returns a Payment OK response.

Observability Framework

Zipkin — :9411

Distributed trace visualization

Kibana — :5601

Log analysis and dashboard

Elasticsearch — :9200

Log indexing and storage

Logstash — :5000

Log aggregation and distribution

ELK Stack and Docker Container Management

1

E — Elasticsearch :9200

It indexes and stores all service logs in real time. It conducts full-text searches on log records formatted in JSON. It serves as the data source for Kibana.

2

L — Logstash: 5000

It gathers and analyzes logs from all services (User, Order, Payment) and transmits them to Elasticsearch.

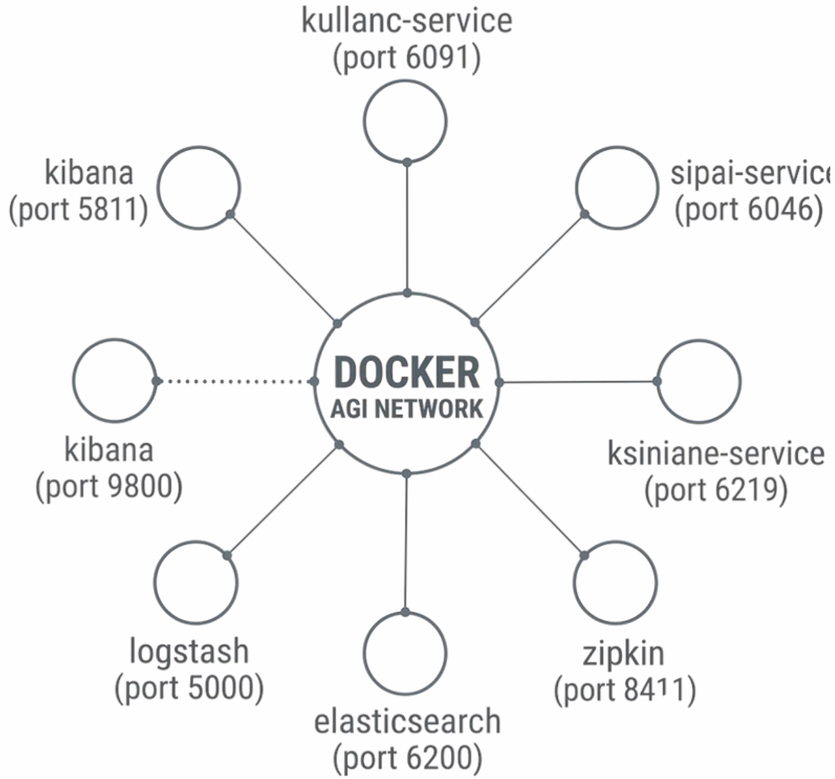
3

K — Kibana :5601

It displays the logs from Elasticsearch within a graphical interface. By utilizing a Trace ID for searching, one can observe the logs of a specific operation across all services on a single screen.

Docker and Container Orchestration

The docker-compose.yml file enables the entire system—including the database, log server, monitoring tools, and applications—to be initiated with a single command.



Container architecture entirely eradicates the "it worked on my computer" dilemma.

Service Architecture and Communication

The user activates a single button, prompting the Order Service (:8082) to function as the orchestrator. The User Service (:8081) conducts the verification, while the Payment Service (:8083) handles the payment processing. Each service operates on distinct ports and interacts via clearly defined endpoints.

Distributed Tracking: Trace Identifier and Span Identifier

Trace Identification

It is a distinctive identification number assigned to a request upon its entry into the system. This ID remains constant, irrespective of the number of services the request traverses. Sleuth embeds this ID in outgoing HTTP headers and transmits it to downstream services. These services interpret this value and incorporate themselves into the same 'Trace'.

a3f2b1c4d5e6f789

Without this ID, associating logs from various services with a single user request becomes impossible.

Span ID

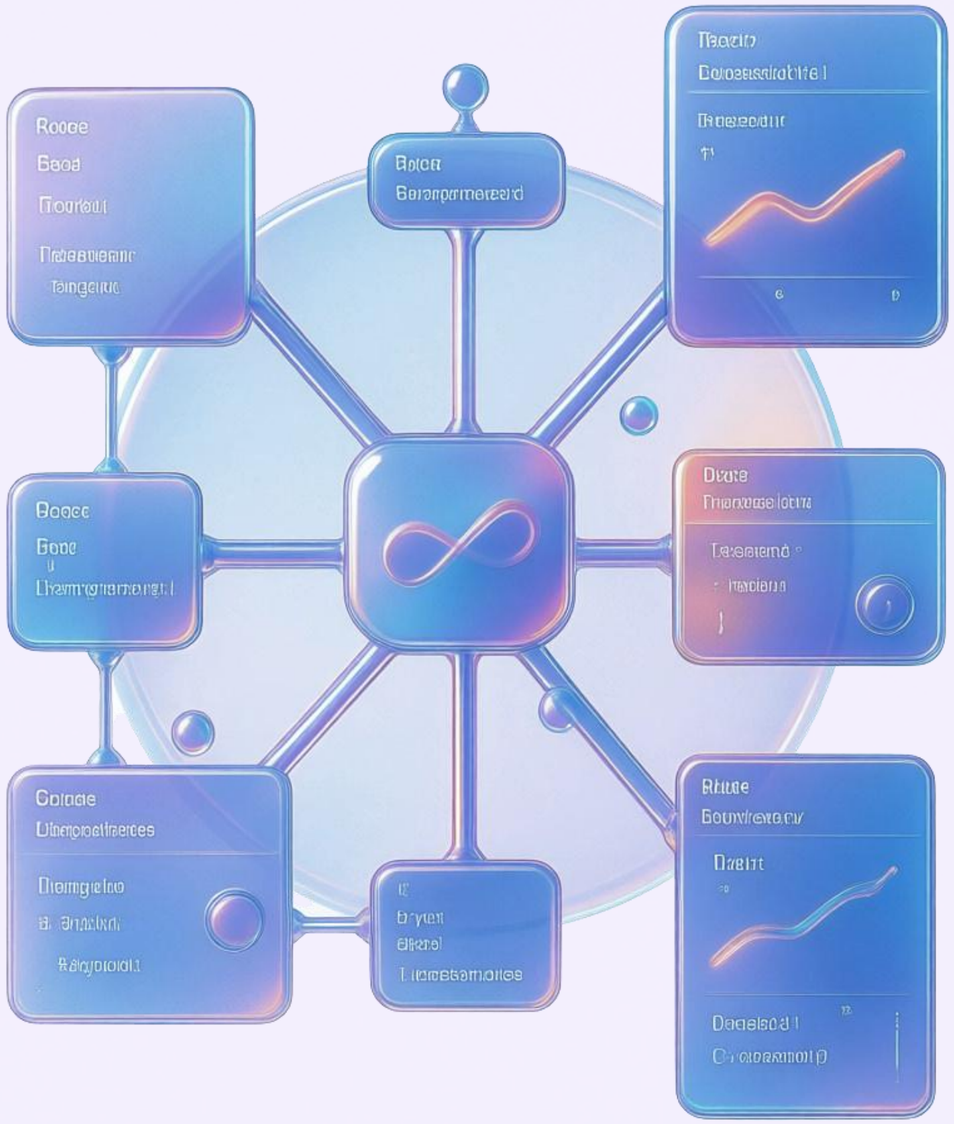
Each service generates a distinct identifier for its operations. A Trace comprises multiple Spans. Each Span contains information regarding its start time, end time, duration, and parent Span.

Example hierarchy:

Order Span → User Span plus Payment Span

This hierarchy is presented in a tree view within Zipkin, allowing for precise visibility into the time each service consumes.

1 : N — Single Trace, 3 Span: Order Service (:8082) /order → User Service (:8081) /user → Payment Service (:8083) /payment. Trace ID: a3f2b1c4d5e6f789 (consistent across all lines — remains unchanged)



How does Spring Cloud Sleuth operate?

Trace ID and Span ID

Each request is allocated a distinct Trace ID upon its entry into the system. This ID is preserved throughout the entire service journey of the request.

Traceability Robustness

In the event of an ordering error, the Trace ID serves to identify the precise millisecond and service where the error transpired.

Request Workflow and Comprehensive Transparency

01

The user selects "Complete Order."

02

The Order Service / order endpoint has been activated.

03

User Service /user is referred to as (RestTemplate)

04

Payment Service /payment is referred to as (RestTemplate)

05

All results are aggregated and presented.

🔗 Trace ID: a3f2b1c4d5e6f789 — Remains consistent across all services. Sleuth automatically transmits the X-B3-TraceId with each RestTemplate invocation.

🔍 Comprehensive Insight

In Zipkin, you can monitor the initiation time of each span, its duration, and whether it resulted in an error on the timeline.

Simplified Debugging

When an error arises, the Trace ID can swiftly pinpoint the specific service and step that encountered the failure.

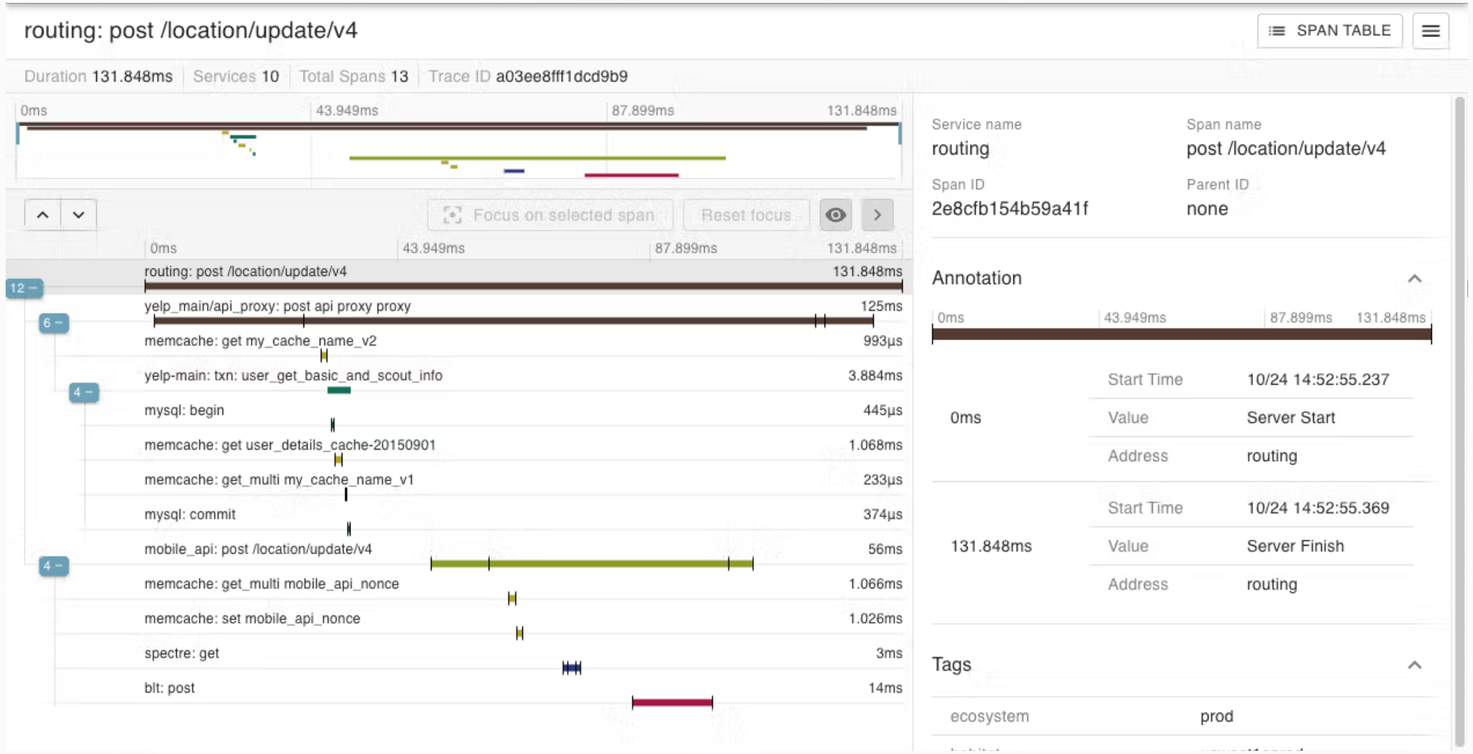
Zipkin: Visual Trace Analysis

What constitutes a speargun?

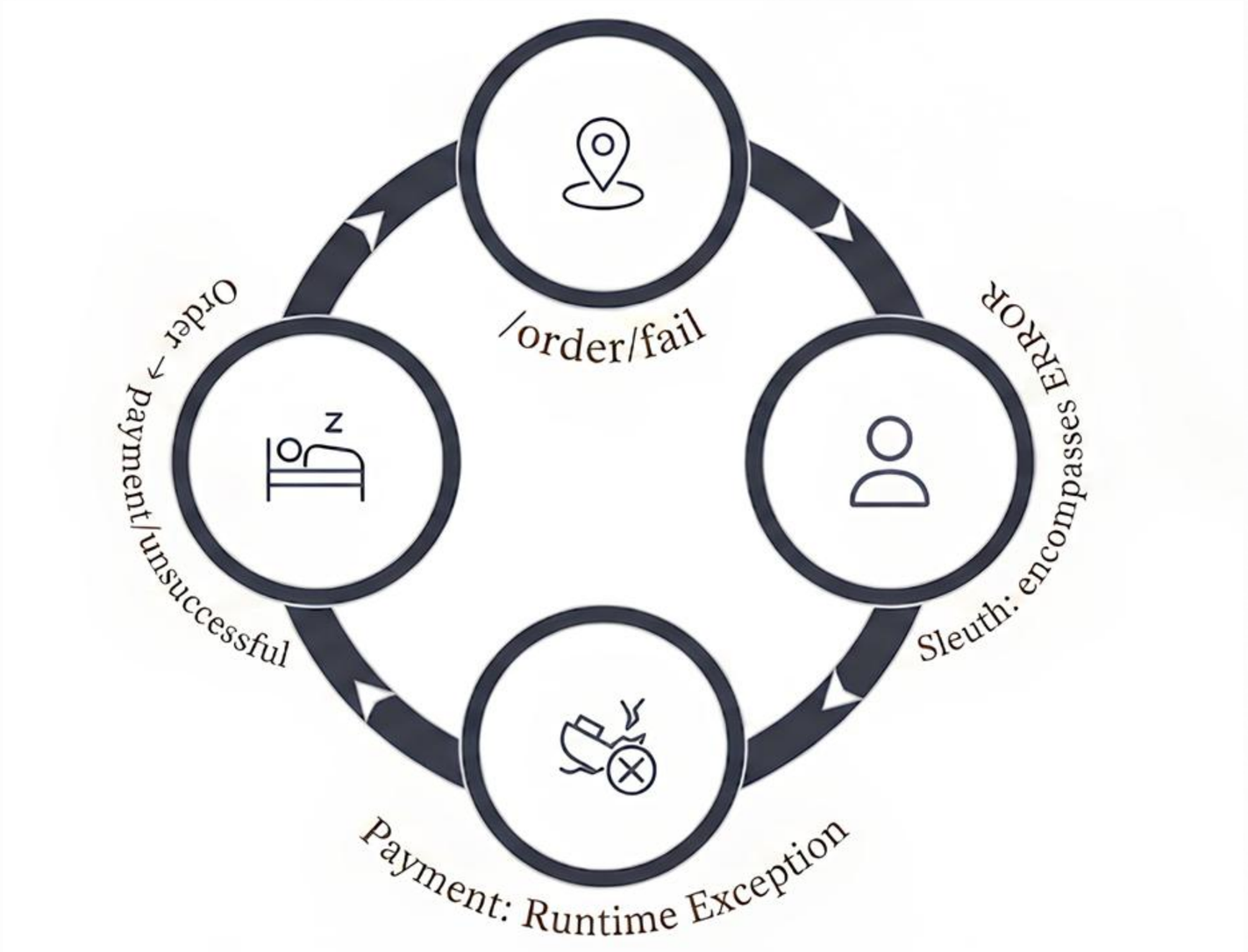
Services transmit finalized Span data to Zipkin through HTTP. Zipkin organizes this data by Trace ID, computes their duration, and displays it in a visual timeline interface.

Features Provided by Zipkin

-Rapid search using Trace ID
-Service-oriented filtering
-Color coding for error and success statuses.
-Comparative Analysis of Span Durations
-Inter-service dependency diagram



Failure Scenarios and Trace Analysis



HTTP Status Codes and Error Management

Scenario: Successful Transaction

The Order Service effectively invokes the User and Payment services, both of which respond with HTTP 200 and the message 'OK'. The Order Controller consolidates these responses and transmits them to the client as
“Order OK | User OK | Payment OK.”
A method that returns a String with `@GetMapping` defaults to HTTP 200 in Spring, requiring no additional annotation.

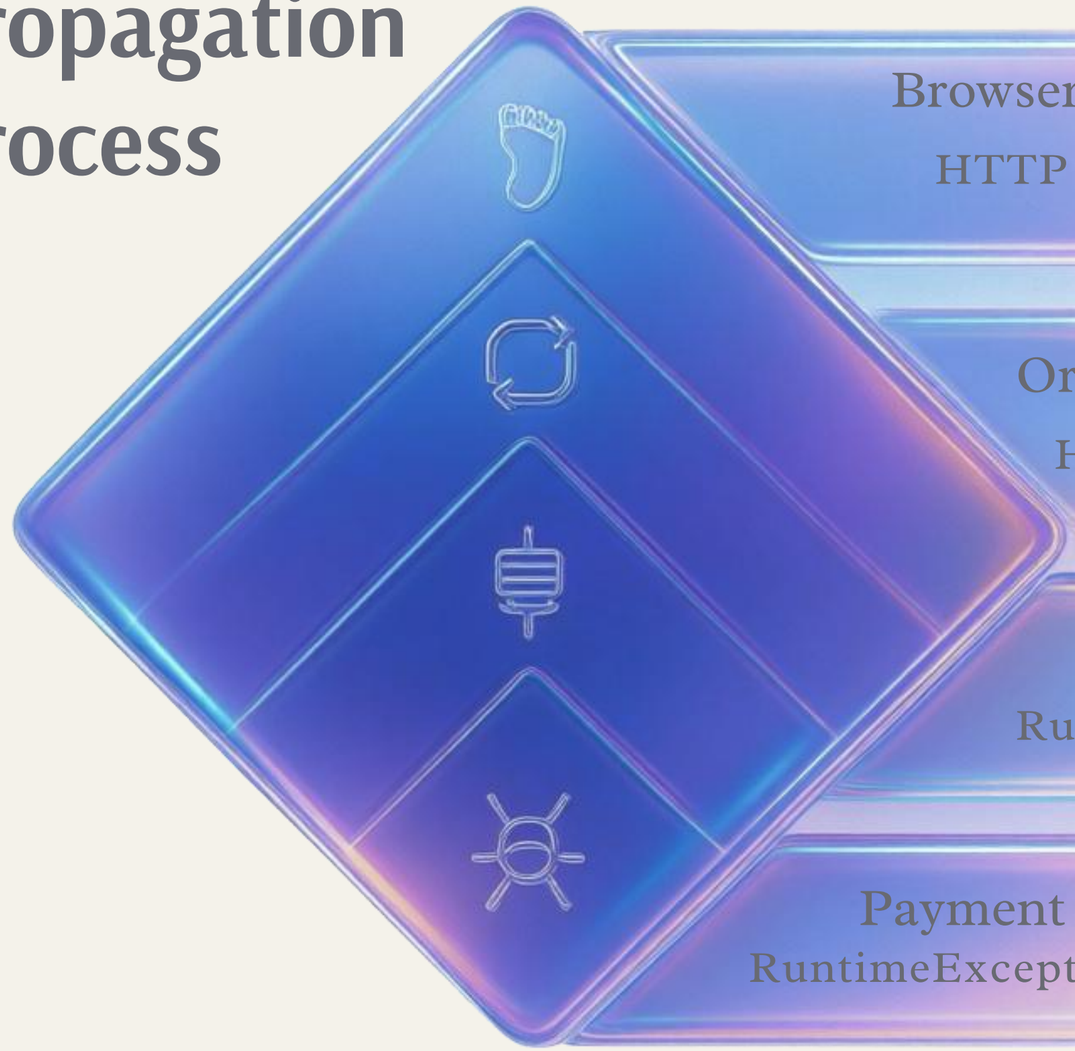
HTTP 200 — OK: All services have returned a successful response.

Scenario: Payment Discrepancy

The payment/fail endpoint generates a RuntimeException. Spring responds to the unhandled exception with an HTTP 500 status. The Order Service captures the
“HttpServerErrorException from Payment using e.getStatusCode()”
And re-throws the identical error. This guarantees that the original error code is conveyed to the client. Sleuth designates both spans as ERROR, and Zipkin presents them in red.

HTTP 500 — Internal Server Error: The error chain is accurately conveyed to the client.

Exception Propagation Process



Browser and Investigator

HTTP 500, two spans ERROR (red)

Order Resent

HttpServerErrorException with e.getStatusCode()

Spring Metamorphosis

RuntimeException → HTTP 500

Payment Issue

RuntimeException in Payment Processing Service

What insights did we gain from ShopTrace?

Distributed Surveillance

With Sleuth and Zipkin, the complete service chain was visualized under a single Trace ID.

Centralized Logging

With the ELK Stack, logs from all services were linked to a Trace ID.

Containerization

Seven containers were initiated with a single command utilizing Docker Compose.

User Interface Design

The single entry point architecture enhanced the user experience.

Error Management

Error scenarios were represented as red traces in Zipkin.

ShopTrace | Spring Boot 3 | Spring Cloud Sleuth | Zipkin | ELK Stack | Docker